Ограничения и CASE

Теория

Ограничения в SQLite.
В SQLite **нельзя добавить ограничение через ALTER TABLE**. Ограничения задаются только при CREATE TABLE. Если нужно добавить CHECK или UNIQUE к уже существующей таблице — придётся пересоздать её.

Что такое CASE.
`CASE` — это условный оператор внутри SQL. Работает как `if-else`.

```
SELECT name,
  CASE
    WHEN price > 100 THEN 'дорого'
    WHEN price > 50 THEN 'средне'
    ELSE 'дёшево'
  END AS price_category
FROM product;
```

Схема работы CASE:

```
price > 100  →  'дорого'
price > 50   →  'средне'
иначе        →  'дёшево'
```

Два вида CASE.

1. **Простой CASE** — сравнивает значение с вариантами:

```
CASE category_id
  WHEN 1 THEN 'Еда'
  WHEN 2 THEN 'Быт'
  ELSE 'Другое'
END
```

2. **Поисковый CASE** — проверяет условия:

```
sql
CASE
  WHEN price > 100 THEN 'дорого'
  WHEN price > 50 THEN 'средне'
  ELSE 'дёшево'
END
```

COALESCE.
Возвращает первое не-NULL значение из списка.

```
SELECT COALESCE(category_id, 0) FROM product;
```

Если `category_id` = NULL, вернёт 0.

NULLIF.
Возвращает NULL, если два значения равны.

```
SELECT 100 / NULLIF(price, 0) FROM product;
```

Если `price` = 0, вернёт NULL, а не ошибку деления на ноль.

CASE в UPDATE.
Можно использовать CASE не только в SELECT, но и в UPDATE:

```
UPDATE product SET price = CASE WHEN price < 0 THEN 0 ELSE price END;
```

CASE в ORDER BY.
Можно задать нестандартный порядок сортировки:

```
SELECT * FROM product
ORDER BY CASE
  WHEN price > 100 THEN 1
  WHEN price > 50 THEN 2
  ELSE 3
END;
```

Типичные ошибки студентов.

1. **Забывают END в CASE.** Синтаксис: `CASE WHEN ... THEN ... ELSE ... END`.
2. **Путают COALESCE и NULLIF.** COALESCE — «первое не-NULL», NULLIF — «NULL, если равны».
3. **Пытаются добавить CHECK через ALTER TABLE.** В SQLite это не работает.
4. **Используют CASE без ELSE.** Если ни одно условие не сработало — вернётся NULL.

Разобранный пример

```sql
-- Создаём таблицу с ограничениями
CREATE TABLE product (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE,
  price REAL CHECK (price >= 0) DEFAULT 0,
  category_id INTEGER
);

-- CASE
SELECT name, price,
  CASE
    WHEN price > 100 THEN 'дорого'
    WHEN price > 50 THEN 'средне'
    ELSE 'дёшево'
  END AS price_category
FROM product;

-- COALESCE
SELECT name, COALESCE(category_id, 0) AS cat FROM product;

-- NULLIF
SELECT name, 1000 / NULLIF(price, 0) AS ratio FROM product;
```

Задачи

1. Создайте таблицу `product` с CHECK на `price >= 0`.
2. Добавьте UNIQUE на `name`.
3. Выведите товары с меткой «дорого/дёшево».
4. Замените NULL в category_id на 0 через COALESCE.
5. Используйте NULLIF для избежания деления на 0.
6. Добавьте DEFAULT на price = 0.
7. CASE с тремя условиями: цена < 50 — «дёшево», 50–100 — «средне», > 100 — «дорого».
8. COALESCE с двумя столбцами: COALESCE(price, 0) + COALESCE(discount, 0).

Задача «со звёздочкой»:

9. Используйте CASE внутри ORDER BY для нестандартной сортировки (например, сначала «дорого», потом «средне», потом «дёшево»).

Задача на поиск ошибки:

```sql
SELECT name, CASE WHEN price > 100 THEN 'дорого' FROM product;
```

Что не так?

Задача на предсказание результата:

```sql
SELECT COALESCE(NULL, NULL, 5);
```

Продвинутый случай:
Используйте CASE в UPDATE:

```sql
UPDATE product SET price = CASE WHEN price < 0 THEN 0 ELSE price END;
```